

# lab02

March 12, 2026

## 1 Discrete convolution

$$z[n] = (x * y)[n] = \sum_{k=-\infty}^{\infty} x[k] y[n-k]$$

### 1.0.1 Example

$$x[n] = \{\mathbf{1}, 2, 3, 4, 5\}, \quad y[n] = \{-1, \mathbf{2}, 1\} \quad (\text{bold} \rightarrow n = 0)$$

| 0 | 0 | 1  | 2  | 3  | 4  | 5  | 0  | 0  | $z[n]$ |
|---|---|----|----|----|----|----|----|----|--------|
| 1 | 2 | -1 | 0  | 0  | 0  | 0  | 0  | 0  | -1     |
| 0 | 1 | 2  | -1 | 0  | 0  | 0  | 0  | 0  | 0      |
| 0 | 0 | 1  | 2  | -1 | 0  | 0  | 0  | 0  | 2      |
| 0 | 0 | 0  | 1  | 2  | -1 | 0  | 0  | 0  | 4      |
| 0 | 0 | 0  | 0  | 1  | 2  | -1 | 0  | 0  | 6      |
| 0 | 0 | 0  | 0  | 0  | 1  | 2  | -1 | 0  | 14     |
| 0 | 0 | 0  | 0  | 0  | 0  | 1  | 2  | -1 | 5      |

In matrix form:

$$(x * y) = \begin{bmatrix} -1 & 0 & 0 & 0 & 0 \\ 2 & -1 & 0 & 0 & 0 \\ 1 & 2 & -1 & 0 & 0 \\ 0 & 1 & 2 & -1 & 0 \\ 0 & 0 & 1 & 2 & -1 \\ 0 & 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \\ 2 \\ 4 \\ 6 \\ 14 \\ 5 \end{bmatrix}$$

### 1.0.2 How to code it:

$$z[n] = (x * y)[n] = \sum_{k=-\infty}^{\infty} \underbrace{x[k]}_{0 \leq k < N} \underbrace{y[n-k]}_{0 \leq n-k < M} \quad n \in \mathbb{Z}$$

$$\underbrace{0 \leq k < N}_{x[k]}, \quad \underbrace{0 \leq n-k < M}_{y[n-k]} \quad (*)$$

$$0 \leq k < N, \quad n - M < k \leq n$$

$$\max(0, n - (M - 1)) \leq k \leq \min(n, N - 1) \quad (**)$$

$$z[n] = \sum_{k=\max(0, n-(M-1))}^{\min(n, N-1)} x[k] y[n-k] \quad 0 \leq n < N + M \quad (***)$$

```
[4]: x = [1, 2, 3, 4, 5]
y = [-1, 2, 1]

# version 1: let's not worry about limits too early
for n in range(-10, 10): # explicit Python loop
    s = 0
    for k in range(-10, 10): # explicit Python loop
        if k >= len(x) or n - k >= len(y) or k < 0 or n - k < 0: # according to
            ↪ conditions in (*)
            continue
        else:
            s += x[k]*y[n-k]
    print(s, end=",")
```

0,0,0,0,0,0,0,0,0,0,-1,0,2,4,6,14,5,0,0,0,

```
[6]: x = [1, 2, 3, 4, 5]
y = [-1, 2, 1]

# version 2: implementing (**) and (***)
for n in range(len(x) + len(y) - 1): # explicit Python loop
    s = 0
    for k in range(max(n - len(y) + 1, 0), min(n + 1, len(x))): # explicit Python
        ↪ loop
        s += x[k]*y[n-k]
    print(s, end=",")
```

-1,0,2,4,6,14,5,

```
[10]: # Using numeric Python
import numpy as np
x = np.array([1, 2, 3, 4, 5])
y = np.array([-1, 2, 1])

# version 3: one explicit loop + vectorized operation
for n in range(len(x) + len(y) - 1): # explicit Python loop
```

```
k = np.arange(max(n - len(y)+1, 0), min(n+1, len(x))) # vectorized op.
print((x[k]*y[n-k]).sum(), end=",") # vectorized op.
```

-1,0,2,4,6,14,5,

```
[24]: # Using numeric Python
import numpy as np
x = np.array([1, 2, 3, 4, 5])
y = np.array([-1, 2, 1])

# one liners with one explicit loop + vectorized op.

# version 4
print([(x[np.arange(max(n - len(y)+1, 0), min(n+1, len(x)))]*y[n-
↪arange(max(n - len(y)+1, 0), min(n+1, len(x)))]).sum() for n in range(len(x)
↪+ len(y) - 1)])

# version 5
print([(lambda k: x[k]*y[n-k])(np.arange(max(n - len(y)+1, 0), min(n+1,
↪len(x))))).sum() for n in range(len(x) + len(y) - 1)])
```

[-1, 0, 2, 4, 6, 14, 5]

[-1, 0, 2, 4, 6, 14, 5]

```
[30]: # convolution using the matrix multiplication approach
# applying only vectorized operations
n, m = len(x), len(y)
i, j = np.arange(n+m-1), np.arange(n)
i, j
```

```
[30]: (array([0, 1, 2, 3, 4, 5, 6]), array([0, 1, 2, 3, 4]))
```

```
[31]: I, J = np.meshgrid(j, i)
I, J
```

```
[31]: (array([[0, 1, 2, 3, 4],
              [0, 1, 2, 3, 4],
              [0, 1, 2, 3, 4],
              [0, 1, 2, 3, 4],
              [0, 1, 2, 3, 4],
              [0, 1, 2, 3, 4],
              [0, 1, 2, 3, 4]]),
array([[0, 0, 0, 0, 0],
       [1, 1, 1, 1, 1],
       [2, 2, 2, 2, 2],
       [3, 3, 3, 3, 3],
       [4, 4, 4, 4, 4],
       [5, 5, 5, 5, 5]]))
```

```
[6, 6, 6, 6, 6]))
```

```
[32]: tmp = J-I  
      tmp
```

```
[32]: array([[ 0, -1, -2, -3, -4],  
            [ 1,  0, -1, -2, -3],  
            [ 2,  1,  0, -1, -2],  
            [ 3,  2,  1,  0, -1],  
            [ 4,  3,  2,  1,  0],  
            [ 5,  4,  3,  2,  1],  
            [ 6,  5,  4,  3,  2]])
```

```
[33]: tmp[tmp >= m] = -1  
      tmp
```

```
[33]: array([[ 0, -1, -2, -3, -4],  
            [ 1,  0, -1, -2, -3],  
            [ 2,  1,  0, -1, -2],  
            [-1,  2,  1,  0, -1],  
            [-1, -1,  2,  1,  0],  
            [-1, -1, -1,  2,  1],  
            [-1, -1, -1, -1,  2]])
```

```
[34]: ind = np.where(tmp >= 0)  
      ind
```

```
[34]: (array([0, 1, 1, 2, 2, 2, 3, 3, 3, 4, 4, 4, 5, 5, 6]),  
      array([0, 0, 1, 0, 1, 2, 1, 2, 3, 2, 3, 4, 3, 4, 4]))
```

```
[44]: H = np.zeros([n + m - 1, n], dtype=int) # without 'dtype' it will use float  
      H[ind]=y[tmp[ind]]  
      H
```

```
[44]: array([[ -1,  0,  0,  0,  0],  
            [  2, -1,  0,  0,  0],  
            [  1,  2, -1,  0,  0],  
            [  0,  1,  2, -1,  0],  
            [  0,  0,  1,  2, -1],  
            [  0,  0,  0,  1,  2],  
            [  0,  0,  0,  0,  1]])
```

```
[45]: np.mat(H)*np.mat(x).T # multiplying as matrices
```

```
[45]: matrix([[ -1],  
            [  0],  
            [  2],  
            [  4],
```

```

[ 6],
[14],
[ 5]])

```

```
[46]: np.array(np.mat(H)*np.mat(x).T).flatten()
```

```
[46]: array([-1,  0,  2,  4,  6, 14,  5])
```

```
[48]: def convolve(x, y):
      """
      using version 3 : one explicit loop + vectorized operation
      """
      z = np.zeros(len(x) + len(y) - 1)
      for n in range(len(z)):# explicit Python loop
          k = np.arange(max(n - len(y)+1, 0), min(n+1, len(x))) # vectorized op.
          z[n] = (x[k]*y[n-k]).sum() # vectorized op.
      return z

```

```
[49]: convolve(x, y)
```

```
[49]: array([-1.,  0.,  2.,  4.,  6., 14.,  5.])
```

```
[51]: N = 10
      impulse = np.zeros(N)
      impulse[0] = 1
      impulse

```

```
[51]: array([1., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

```
[52]: convolve(x, impulse)
```

```
[52]: array([1., 2., 3., 4., 5., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

```
[53]: convolve(impulse, x)
```

```
[53]: array([1., 2., 3., 4., 5., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

```
[55]: step = np.zeros(N)
      step[:N//2] = 1
      step

```

```
[55]: array([1., 1., 1., 1., 1., 0., 0., 0., 0., 0.])
```

```
[56]: convolve(step, step)
```

```
[56]: array([1., 2., 3., 4., 5., 4., 3., 2., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])

```

```
[69]: import matplotlib.pyplot as plt
n = np.arange(N) - N//2
nc = np.arange(2*N - 1) - N
plt.stem(nc, convolve(step[n], step[n]), linefmt='b-', markerfmt='ro',
        ↪basefmt='k-')
```

[69]: <StemContainer object of 3 artists>

